Doomscroll

Test Cases and Testing Results — Lab 4

Version 1.0
Prepared by: Project Team -- SC2006
Date: 18 October 2026

1. Scope and Approach

Per the Lab 4 manual (3.2), this document designs black-box test cases (equivalence-class partitioning and boundary-value analysis) against one important control class, and white-box test cases (basis-path testing) against two methods implementing complex decision logic. Test cases are kept to the minimum needed to cover each identified equivalence class, boundary, and decision path.

Execution: every case below has been run against the real implementation and its "Actual Output" filled in -- either directly (tests/test_scroll_trigger.py, tests/test_gaze_content.py, and the new tests/test_attention_gate.py, all run by the project team with pytest, all green) or, for the handful of specific timing setups a named pytest case doesn't cover exactly, by driving the real classes with the same inputs by hand and recording what they actually returned. One case (TC-ST-11) turned up a real authoring mistake in this document's own original Expected Output, corrected below with an explanation.

2. Black-Box Testing: ScrollTrigger.update()

Class under test: ScrollTrigger (src/doomscroll/fixation/scroll_trigger.py). Chosen as the "one important control class" because it is the final decision point for every Autoscroll action (Use Case 4.2) and combines three independent trigger tiers with timing state (dwell, cooldown).

Signature: update(fy, blocks, gaze_point, viewport_bottom, timestamp_ms, content_available) -> (bool, str). Relevant constants: BOTTOM_DEFINITE_FRACTION = 0.85, FAST_DWELL_MS = 150, CONTENT_TIER_DWELL_MS = 1000, COOLDOWN_MS = 800.

2.1 Equivalence Classes

ID	Equivalence Class	Representative Condition
EC1	fy (fraction of window height) in the definite bottom zone	fy >= 0.85
EC2	fy above the definite bottom zone	fy < 0.85
EC3	Content available and gaze has reached end of lowest block	content_available=True, reached_end=True

EC4	Content available but gaze has not reached end of any block	content_available=True, reached_end=False
EC5	Content not available (no OCR backend / tier 1 only)	content_available=False
EC6	Within cooldown window since last scroll fired	elapsed since note_scroll_fired() < COOLDOWN_MS
EC7	Outside cooldown window	elapsed >= COOLDOWN_MS

2.2 Boundary Values

ID	Boundary	Value Tested
BV1	fy exactly at BOTTOM_DEFINITE_FRACTION	fy = 0.85
BV2	fy just below the boundary	fy = 0.849
BV3	fy just above the boundary	fy = 0.851
BV4	Dwell time exactly at FAST_DWELL_MS	dwell = 150 ms
BV5	Dwell time 1 ms short of FAST_DWELL_MS	dwell = 149 ms
BV6	Dwell time exactly at CONTENT_TIER_DWELL_MS	dwell = 1000 ms
BV7	Cooldown exactly at COOLDOWN_MS since last fire	elapsed = 800 ms
BV8	Cooldown 1 ms short of COOLDOWN_MS	elapsed = 799 ms

2.3 Test Cases (combining EC + BV coverage)

Test ID	Test Input / Setup	Expected Output	Actual Output
TC-ST-01	No cooldown active; fy=0.90 held for 150ms; content unavailable	(True, 'SCROLL (bottom15)')	(True, 'SCROLL (bottom15)') -- MATCH
TC-ST-02	No cooldown active; fy=0.80 held for 150ms; content unavailable	(False, 'not scrolling') (tier 1 not met, no content tiers available)	(False, 'not scrolling') -- MATCH
TC-ST-03	fy=0.85 exactly, held for 150ms (BV1)	(True, 'SCROLL (bottom15)') (>= is inclusive)	(True, 'SCROLL (bottom15)') -- MATCH
TC-ST-04	fy=0.849, held for 150ms (BV2)	(False, 'not scrolling') (just below threshold)	(False, 'not scrolling') -- MATCH
TC-ST-05	fy=0.90 held for only 149ms (BV5)	(False, 'bottom15 149/150ms') (dwell not yet satisfied)	(False, 'bottom15 149/150ms') -- MATCH
TC-ST-06	fy=0.90 held for	(True, 'SCROLL	(True, 'SCROLL

	exactly 150ms (BV4)	(bottom15)')	(bottom15)') -- MATCH
TC-ST-07	Content available, has_reached_end()=True, held for 150ms, fy below 0.85	(True, 'SCROLL (reached_end)')	(True, 'SCROLL (reached_end)') -- MATCH
TC-ST-08	Content available, should_scroll()=True (tier 3), has_reached_end()=False, held for only 999ms	(False, 'content_ok 999/1000ms') (content-tier dwell not yet satisfied)	(False, 'content_ok 999/1000ms') -- MATCH
TC-ST-09	Content available, has_reached_end()=False, should_scroll()=False, fy below 0.85	(False, 'not scrolling') (no tier satisfied)	(False, 'not scrolling') -- MATCH
TC-ST-10	Tier 1 condition met this tick, but note_scroll_fired() was called 799ms earlier (BV8)	(False, 'cooldown...') (within cooldown)	(False, 'cooldown...') -- MATCH
TC-ST-11	Tier 1 condition met on the single tick exactly 800ms after note_scroll_fired() (BV7)	(False, 'bottom15 0/150ms') -- corrected; see note below	(False, 'bottom15 0/150ms') -- MATCH (corrected expectation)
TC-ST-12	fy=0.90 held for 100ms, then reset_dwell() called, then fy=0.90 again for only 100ms (no cooldown)	(False, 'bottom15 100/150ms') (dwell timer restarted by reset, not carried over)	(False, 'bottom15 100/150ms') -- MATCH

Cross-reference: TC-ST-01 through TC-ST-06, and TC-ST-10/12, correspond closely to the existing cases in tests/test_scroll_trigger.py and are confirmed by its 12/12 pytest pass. TC-ST-07/08/09 and TC-ST-11 were additionally driven by hand against the real ScrollTrigger (same inputs, reproduced in Part 2 of lab3/ai_project_init_report.pdf's sibling exercise report, lab4/coding_agent_report.pdf) since they use setups the existing suite doesn't exercise with these exact numbers.

Note on TC-ST-11: this document's own original Expected Output for this row -- (True, 'SCROLL (bottom15)'), on the theory that the tier-1 condition being met "this tick" as cooldown lapses should fire right away -- was wrong, and the mistake is instructive. The real ScrollTrigger.update() checks cooldown before any dwell/tier logic and resets dwell state (_dwell_start_ms, _dwell_tier) on every tick spent in cooldown, so no dwell can accumulate while cooldown is still active; the very first tick after cooldown clears always starts a brand-new dwell at 0ms elapsed, same as TC-ST-01's first tick. Firing on that first post-cooldown tick would require the class to pre-accumulate dwell during

cooldown, which it deliberately does not do. (A second tick 150ms later, as in TC-ST-01/03/06's two-tick pattern, does fire -- verified separately.) This is the same cooldown/dwell-reset ordering examined in lab4/coding_agent_report.pdf's Exercise 1, where an AI asked to regenerate this class from scratch got this exact detail wrong in the other direction (it let dwell accumulate through cooldown); this test case shows the mistake is easy enough to make by hand too, not just by an AI working from a spec.

3. White-Box Testing: Basis-Path Coverage

Two methods implementing non-trivial decision logic were chosen, per the manual's requirement to apply basis-path testing to two methods.

3.1 ContentGazeTracker.update() (src/doomscroll/content/gaze_content.py)

update(blocks, gaze_point, timestamp_ms) returns nothing; it only mutates internal state (_current_id, _current_since_ms, _gap_since_ms, _seen_ids). Each test case therefore drives update() for a short sequence of ticks and observes the effect through the public methods has_reached_end() / should_scroll() and the seen_ids property, which is legitimate for a white-box test since the private state is known from the source. Its independent paths (determined by the nested hit_id == current_id / current_id is None / gap_since_ms branches) and a minimal test case per path:

Path	Condition	Expected Behaviour	Test Input / Setup	Expected Output	Actual Output
P1	hit_id == current_id, both None (gaze on nothing, still on nothing)	No dwell change; seen_ids unaffected.	TC-CGT-01: no prior target; gaze 200px from every block (outside HIT_TEST_MAX_DISTANCE_PX=35), two ticks	block never added to seen_ids	seen_ids stayed empty across both ticks -- MATCH
P2	hit_id == current_id, both equal a real block id, dwell not yet at _required_dwell_ms(block)	Dwell continues; block not yet marked seen.	TC-CGT-02: target=A already current; gaze stays within 10px of A for 100ms (< required dwell for A's width)	A not in seen_ids after this tick	A not in seen_ids at 100ms (required dwell for this A is 200ms) -- MATCH
P3	hit_id == current_id, both equal a real block id,	Block is added to seen_ids.	TC-CGT-03: target=A already current since	A is in seen_ids after this tick	A in seen_ids at t=200ms (== required dwell) --

	dwell reaches _required_dwell_ms(block) on this tick		t=0; gaze stays within 10px of A; this update() call is at t = _required_dwell_ms(A)		MATCH
P4	current_id is None (no prior target); hit found within HIT_TEST_MAX_DISTANCE_PX	Commit immediately -- no grace period, since there is no existing dwell progress to protect.	TC-CGT-04: no prior target; gaze lands 10px from block A	current target becomes A, dwell starts at this tick's timestamp (0ms elapsed, not yet seen)	_current_id='a', _current_since_ms=0, seen_ids empty -- MATCH
P5	Diverging from a real target (hit_id != current_id, current_id not None); first tick of the divergence (_gap_since_ms was None)	_gap_since_ms is set to this tick; current target and its dwell progress are unchanged.	TC-CGT-05: target=A current with dwell in progress; this tick's gaze is 200px from every block (no hit) for the first time	target remains A, A's prior dwell progress preserved	_current_id still 'a', _current_since_ms unchanged, _gap_since_ms set -- MATCH
P6	Diverging continues; elapsed since _gap_since_ms is still < GAP_GRACE_MS (500ms)	No change yet -- neither reverts nor commits the switch.	TC-CGT-06: continuing from P5, 300ms after the divergence began (< 500ms)	target still A, A's dwell progress still preserved	_current_id still 'a', _gap_since_ms unchanged at the P5 tick -- MATCH
P7	Diverging continues; elapsed since _gap_since_ms reaches GAP_GRACE_MS (500ms)	The switch commits: target becomes the new hit (or None), dwell resets.	TC-CGT-07: continuing from P5, 500ms after the divergence began	target becomes the new hit_id (or None if still no block within range), dwell restarts at 0	_current_id became None (still no block in range), _current_since_ms reset to this tick, _gap_since_ms cleared -- MATCH

All seven paths were confirmed both by tests/test_gaze_content.py's 23/23 pytest pass (which exercises this same state machine, including its own, corrected,

test_default_dwell_has_a_floor_even_for_a_near_zero_width_block case) and by driving ContentGazeTracker directly with each row's exact inputs by hand.

3.2 AttentionGate.update() (src/doomscroll/fixation/attention_gate.py)

This method decides whether Autoscroll should be paused, based on whether the current Head Pose's yaw/pitch exceed configured limits, with an asymmetric grace period: pausing requires the out-of-range condition to persist for pause_grace_ms, but resuming is immediate once the pose returns within range. Its independent paths:

Path	Condition	Expected Behaviour	Test Input / Setup	Expected Output	Actual Output
P1	Not currently paused; head pose within max_yaw_deg/max_pitch_deg	Remain active (not paused).	TC-AG-01: yaw=10°, pitch=5°, limits 35°/30°	paused=False	paused=False -- MATCH
P2	Not currently paused; head pose exceeds a limit, first frame out of range	Remain active; grace timer starts.	TC-AG-02: yaw=40° (>35°), first frame	paused=False	paused=False -- MATCH
P3	Not currently paused; head pose has exceeded a limit for less than pause_grace_ms	Remain active; grace timer still running.	TC-AG-03: yaw=40° held 300ms (< 500ms grace)	paused=False	paused=False -- MATCH
P4	Not currently paused; head pose has exceeded a limit for >= pause_grace_ms	Transition to paused.	TC-AG-04: yaw=40° held 500ms	paused=True	paused=True -- MATCH
P5	Currently paused; head pose still outside limits	Remain paused.	TC-AG-05: paused=True, yaw=40° this frame	paused=True	paused=True -- MATCH
P6	Currently paused; head pose returns within limits	Resume immediately (no grace period on resume).	TC-AG-06: paused=True, yaw=10°, pitch=5° this frame	paused=False	paused=False -- MATCH

Confirmed by `pytest tests/test_attention_gate.py -v`: 9 passed (TC-AG-01 through TC-AG-06 above, plus three supplementary cases beyond this table -- pitch-only trigger, a missing head pose, and an intermittent in-range frame correctly resetting the grace timer).